A

CSE221
Summer 2026
Quiz - 01
Duration: 30 Minutes
Marks: 20

Name:	ID:	Section:

Question 1 [10 Marks]

a. Consider the following five functions.

i) $h_1(n) = (\log n)^5$	ii) $h_2(n) = n \cdot \sqrt{n}$	iii) $h_3(n) = n^2 \log n$	iv) $h_4(n) = n^3$	v) $h_5(n) = 2^n$

Write a correct asymptotic tight bound (Θ) for each function above, then sort all five functions in ascending order of their growth rate, assuming n is significantly large. Just write the sorted order — no simulation required. **[2]**

b. Analyze the following code and determine its overall time complexity in Big-O notation. Clearly show how many times each loop runs before combining them. **[4]**

```
int result = 0;
for (int i = 1; i <= n; i++) {
   for (int j = 1; j <= i; j *= 2) {
      result++;
   }
}
return result;
```

c. Consider the following code.

```
int solve(int n) {
   if (n <= 2) {
      return 1;
   }
   int result = solve((int)sqrt(n));
   return result + 1;
}
```

Write the recurrence relation T(n) for the function above, then solve it for its time complexity using any suitable method (recursion tree, substitution, or master theorem). Show your work. **[4]**

Question 2 [10 Marks]

a. You need to sort the exam scores (integers from 0 to 100) of 2 million students as fast as possible. Memory is not a constraint. Name the most suitable sorting algorithm for this scenario and state its time complexity. Justify your choice in one or two lines. **[2]**

b. Consider the array [10, 4, 15, 2, 8, 20, 6]. Using Quick Sort with the first element of the array as the pivot, show the full step-by-step simulation of the algorithm until the entire array is sorted. Clearly highlight the pivot selection and the resulting array after each partitioning step. **[6]**

c. Is Insertion Sort a stable sorting algorithm? Justify your answer with a short example. **[2]**

B

CSE221
Summer 2026
Quiz - 01
Duration: 30 Minutes
Marks: 20

Name:	ID:	Section:

Question 1 [10 Marks]

a. Consider the following five functions.

i) $k_1(n) = (\log n)^7$	ii) $k_2(n) = n \log n$	iii) $k_3(n) = n^2$	iv) $k_4(n) = n^{2.5}$	v) $k_5(n) = 3^n$

Write a correct asymptotic tight bound (Θ) for each function above, then sort all five functions in ascending order of their growth rate, assuming n is significantly large. Just write the sorted order — no simulation required. **[2]**

b. Analyze the following code and determine its overall time complexity in Big-O notation. Clearly show how many times each loop runs before combining them. **[4]**

```
int result = 0;
for (int i = 1; i <= n; i *= 2) {
   for (int j = 0; j < n; j++) {
      result += i + j;
   }
}
return result;
```

c. Consider the following code.

```
int solve(int n) {
   if (n <= 2) {
      return 1;
   }
   int result = solve((int)sqrt(n));
   return result + 1;
}
```

Write the recurrence relation T(n) for the function above, then solve it for its time complexity using any suitable method (recursion tree, substitution, or master theorem). Show your work. **[4]**

Question 2 [10 Marks]

a. You need to sort the ages (integers from 0 to 120) of 5 million people as fast as possible. Memory is not a constraint. Name the most suitable sorting algorithm for this scenario and state its time complexity. Justify your choice in one or two lines. **[2]**

b. Consider the array [12, 5, 18, 3, 9, 25, 7]. Using Quick Sort with the first element of the array as the pivot, show the full step-by-step simulation of the algorithm until the entire array is sorted. Clearly highlight the pivot selection and the resulting array after each partitioning step. **[6]**

c. Is Selection Sort a stable sorting algorithm? Justify your answer with a short example.**[2]**

CSE221

Quiz 1

Set A Solutions

Question 1 [10 Marks]

a. Asymptotic Bounds and Sorting [3]
First, we establish the tight bound (Θ) for each function:

- $h_1(n) = (\log n)^5 \Longrightarrow \Theta((\log n)^5)$
- $h_2(n) = n \cdot \sqrt{n} = n^{1.5} \Longrightarrow \Theta(n^{1.5})$
- $h_3(n) = n^2 \log n \Longrightarrow \Theta(n^2 \log n)$
- $h_4(n) = n^3 \Longrightarrow \Theta(n^3)$
- $h_5(n) = 2^n \Longrightarrow \Theta(2^n)$

Sorting in ascending order of growth rate:
$h_1(n) \leq h_2(n) \leq h_3(n) \leq h_4(n) \leq h_5(n)$

b. Time Complexity Analysis [3]
The outer loop runs from $i = 1$ to n, incrementing by 1. Thus, it executes exactly n times. The inner loop runs from $j = 1$ to i, multiplying j by 2 in each step. The number of times this inner loop executes for a given i is exactly $\log_2(i)$.

Combining them, the total number of operations is the sum of the inner loop executions over all iterations of the outer loop: $\sum_{i=1}^{n} \log_2(i) = \log_2(1) + \log_2(2) + \cdots + \log_2(n) = \log_2(n!)$

By Stirling's approximation, $\log(n!) \approx n \log n$. Therefore, the overall time complexity is $O(n \log n)$.

c. Recurrence Relation and Solution [4]
The function solve(n) calls itself with the argument $\sqrt{n}$. The rest of the operations (if condition, re turns, variable assignments) take constant time $O(1)$.

Recurrence Relation: $T(n) = T(\sqrt{n}) + 1$
To solve this using the Master Theorem for Dividing Functions, we first perform a variable sub stitution to convert it into the standard format $T(n) = aT(n/b) + f(n)$.
Let $n = 2^m$. Taking the base-2 logarithm of both sides gives $m = \log_2 n$. Substituting 2^m into the recurrence:

$$T(2^m) = T(2^{m/2}) + 1$$

Now, we define a new function $S(m) = T(2^m)$. Replacing this in our equation yields:

$$S(m) = S(m/2) + 1$$

This is now in the standard form $S(m) = aS(m/b) + f(m)$ where $f(m) = O(m^k \log^p m)$. • a =

1

- $b = 2$

- $f(m) = 1$, which means $k = 0$ and $p = 0$ (since $1 = O(m^0 \log^0 m)$)

Next, we evaluate b^k: $2^0 = 1$.
Since $a = b^k (1 = 1)$, we look at the value of p.
Here, $p = 0$, which falls under the condition $p > -1$. According to the Master Theorem, the time complexity is $O(m^{\log_b a} \log^{p+1} m)$:

$$S(m) = O(m^{\log_2 1} \log^{0+1} m) = O(m^0 \log^1 m) = O(\log m)$$

Finally, we substitute $m = \log_2 n$ back into the result to find $T(n)$:

$$T(n) = O(\log(\log n))$$

Question 2 [10 Marks]

a. Sorting Algorithm Choice [2]
Algorithm: Counting Sort.
Time Complexity: $O(N)$ (specifically $O(N + K)$ where N is elements and K is the range). Justification: Counting sort operates in linear time and is perfectly suited here because the range of exam scores ($K = 101$, from 0 to 100) is drastically smaller than the number of students ($N = 2,000,000$), and memory is not a constraint.

b. Quick Sort Simulation [6]
Initial Array: [10, 4, 15, 2, 8, 20, 6]. The pivot is always the first element of the current sub-array.

1. Step 1: Pivot is 10.
   Elements < 10: 4, 2, 8, 6
   Elements > 10: 15, 20
   Array after partitioning: [4, 2, 8, 6, 10, 15, 20]
2. Step 2: Recursive call on left sub-array [4, 2, 8, 6]. Pivot is 4.
   Elements < 4: 2
   Elements > 4: 8, 6
   Array after partitioning: [2, 4, 8, 6, 10, 15, 20]
3. Step 3: Recursive call on left sub-array [2]. Base case reached (already sorted).
4. Step 4: Recursive call on right sub-array [8, 6]. Pivot is 8.
   Elements < 8: 6
   Elements > 8: None
   Array after partitioning: [2, 4, 6, 8, 10, 15, 20]
5. Step 5: Recursive call on right sub-array [15, 20] (from Step 1). Pivot is 15.
   Elements < 15: None
   Elements > 15: 20
   Array after partitioning: [2, 4, 6, 8, 10, 15, 20]

Final sorted array: [2, 4, 6, 8, 10, 15, 20].

c. Insertion Sort Stability [2]
Yes, Insertion Sort is a stable sorting algorithm.
Justification: During insertion, an element is shifted backward only if it is *strictly less* than the element before it. If two elements are equal, the algorithm does not swap them, perfectly preserving their original relative order.
Example: Sorting $[4_A, 3, 4_B] \Longrightarrow [3, 4_A, 4_B]$.

Set B Solutions

Question 1 [10 Marks]

a. Asymptotic Bounds and Sorting [3]
First, we establish the tight bound (Θ) for each function:

- $k_1(n) = (\log n)^7 \implies \Theta((\log n)^7)$
- $k_2(n) = n \log n \implies \Theta(n \log n)$
- $k_3(n) = n^2 \implies \Theta(n^2)$
- $k_4(n) = n^{2.5} \implies \Theta(n^{2.5})$
- $k_5(n) = 3^n \implies \Theta(3^n)$

Sorting in ascending order of growth rate:
$k_1(n) \leq k_2(n) \leq k_3(n) \leq k_4(n) \leq k_5(n)$

b. Time Complexity Analysis [3]
The outer loop runs from $i = 1$ to n, multiplying by 2 each iteration. Thus, it executes exactly $\log_2(n)$ times.
The inner loop runs from $j = 0$ to $n - 1$, incrementing by 1 each time. It executes exactly n times.
Because the inner loop's limit (n) is independent of the outer loop's variable (i), we can simply multiply their executions together. Total operations = $\log_2(n) \times n$.
Therefore, the overall time complexity is $O(n \log n)$.

c. Recurrence Relation and Solution [4]
Same as Set A

Question 2 [10 Marks]

a. Sorting Algorithm Choice [2]
Algorithm: Counting Sort.
Time Complexity: $O(N)$ (specifically $O(N + K)$).
Justification: Counting sort is the most suitable because the dataset is massive ($N = 5,000,000$ people) but the range of integers is exceptionally small and strictly bounded ($K = 121$, from 0 to 120), allowing it to sort in linear time without memory issues.

b. Quick Sort Simulation [6]
Initial Array: [12, 5, 18, 3, 9, 25, 7]. The pivot is always the first element of the current sub-array.

1. Step 1: Pivot is 12.
   Elements < 12: 5, 3, 9, 7
   Elements > 12: 18, 25
   Array after partitioning: [5, 3, 9, 7, 12, 18, 25]
2. Step 2: Recursive call on left sub-array [5, 3, 9, 7]. Pivot is 5.
   Elements < 5: 3
   Elements > 5: 9, 7
   Array after partitioning: [3, 5, 9, 7, 12, 18, 25]
3. Step 3: Recursive call on left sub-array [3]. Base case reached (already sorted).
4. Step 4: Recursive call on right sub-array [9, 7]. Pivot is 9.
   Elements < 9: 7
   Elements > 9: None
   Array after partitioning: [3, 5, 7, 9, 12, 18, 25]

5. Step 5: Recursive call on right sub-array [18, 25] (from Step 1). Pivot is 18.
   Elements < 18: None
   Elements > 18: 25
   Array after partitioning: [3, 5, 7, 9, 12, 18, 25]

Final sorted array: [3, 5, 7, 9, 12, 18, 25].

c. Selection Sort Stability [2]

No, Selection Sort is not a stable sorting algorithm.

Justification: Selection sort works by finding the minimum element in the unsorted portion and swap ping it with the first unsorted element. This blind swap can easily jump over an element of equal value, destroying their original relative order.

Example: Sorting [4_A, 4_B, 2]. The algorithm swaps 2 with 4_A, resulting in [2, 4_B, 4_A]. The relative order of the 4s has been flipped.

A

CSE221
Summer 2026
Quiz - 02
Duration: 30 Minutes
Marks: 20

Name:	ID:	Section:

Question 1 [08 Marks]

Consider an array of N unique values. For some index i, the values are in increasing order from index 0 to i-1 and again from i to N-1. Every value in the first part is greater than every value in the second part. Read the example array below from left to right.

[13, 17, 21, 25, 3, 6, 9, 11]

a. Propose an algorithm to find the index i. Present the solution using pseudocode, programmable code, a flowchart, or step-by-step logical instructions. [3]
b. By showing the necessary mathematics, explain how Karatsuba's Fast Multiplication algorithm converts an N-digit multiplication into three N/2-digit multiplications. [3]
c. Can the same algorithm be used to multiply two N-bit binary numbers? Explain briefly. [2]

Question 2 [07 Marks]

The following undirected, unweighted graph represents corridors between eight rooms.
V = {1, 2, 3, 4, 5, 6, 7, 8}
E = {(1,4), (4,5), (1,2), (2,5), (1,6), (6,3), (2,3), (3,7), (5,7), (3,8)}

a. Give the adjacency-list representation of the graph. [1]
b. Determine whether the graph is bipartite/bicolorable. Show a valid grouping/coloring of the vertices. [3]
c. Show a simulation of BFS, with proper use of the queue and other necessary details, to find a shortest path from Room 4 to Room 6. [3]

Question 3 [05 Marks]

At Level 2 of a strategy game, you may choose one energy box or a block of consecutive boxes without gaps. Positive values increase your health and negative values reduce it.
Energy boxes = [2, -3, 5, -1, 4, -6, 3, 2]
Using the O(N log N) Divide-and-Conquer algorithm, show a proper simulation to find the maximum possible health gain.

B

CSE221
Summer 2026
Quiz - 01
Duration: 30 Minutes
Marks: 20

Name:	ID:	Section:

Question 1 [08 Marks]

Consider an array of N unique values. For some index i, the values are in increasing order from index 0 to i-1 and again from i to N-1. Every value in the first part is greater than every value in the second part. Read the example array below from left to right.

[13, 17, 21, 25, 3, 6, 9, 11]

a. Propose an algorithm to find the index i. Present the solution using pseudocode, programmable code, a flowchart, or step-by-step logical instructions. [3]
b. By showing the necessary mathematics, explain how Karatsuba's Fast Multiplication algorithm converts an N-digit multiplication into three N/2-digit multiplications. [3]
c. Can the same algorithm be used to multiply two N-bit binary numbers? Explain briefly. [2]

Question 2 [07 Marks]

The following undirected, unweighted graph represents corridors between eight rooms.
V = {1, 2, 3, 4, 5, 6, 7, 8}
E = {(6,2), (6,3), (2,7), (7,1), (2,5), (5,1), (5,4), (4,8), (1,8), (3,8)}

a. Give the adjacency-list representation of the graph. [1]
b. Determine whether the graph is bipartite/bicolorable. Show a valid grouping/coloring of the vertices. [3]
c. Show a simulation of BFS, with proper use of the queue and other necessary details, to find a shortest path from Room 4 to Room 6. [3]

Question 3 [05 Marks]

At Level 2 of a strategy game, you may choose one energy box or a block of consecutive boxes without gaps. Positive values increase your health and negative values reduce it.
Energy boxes = [-2, 4, -1, 3, -5, 6, 2, -4]
Using the O(N log N) Divide-and-Conquer algorithm, show a proper simulation to find the maximum possible health gain.

CSE221 Summer 2026 – Quiz 2 Solutions

Set A

Question 1

(a) The required split i is the index of the minimum value.

```
FindSplit(A, n)
  input: nonempty rotated increasing array A[0 .. n − 1]
  l ← 0; r ← n − 1
  while l < r do
    m ← l + ⌊(r − l)/2⌋
    if A[m] > A[r] then
      l ← m + 1    // minimum is right of m
    else
      r ← m    // minimum is at m or to its left
  return l
```

The invariant is that $i \in [l, r]$; every iteration halves this interval. For the given array,

$$(l, r, m): \ (0, 7, 3) \to (4, 7, 5) \to (4, 5, 4) \to (4, 4, -),$$

so $\boxed{i = 4}$ and the time is $\mathcal{O}(\log n)$.

(b) Split at $m = \lceil N/2 \rceil$ digits:

$$x = aB^m + b, \qquad y = cB^m + d.$$

Ordinary expansion gives

$$xy = acB^{2m} + (ad + bc)B^m + bd,$$

which appears to need four half-size products. Karatsuba computes only

$$P = ac, \qquad Q = bd, \qquad R = (a + b)(c + d).$$

Because $R = P + ad + bc + Q$, the middle term is $ad + bc = R - P - Q$. Hence

$$\boxed{xy = PB^{2m} + (R - P - Q)B^m + Q}.$$

Thus $T(N) = 3T(N/2) + \Theta(N) = \Theta(N^{\log_2 3})$.

(c) Yes. Put $B = 2$; multiplication by 2^m is a bit shift.

Question 2

(a)

1	$2, 4, 6$	5	$2, 4, 7$
2	$1, 3, 5$	6	$1, 3$
3	$2, 6, 7, 8$	7	$3, 5$
4	$1, 5$	8	3

(b) Bipartite: $\boxed{L = \{1, 3, 5\},\ R = \{2, 4, 6, 7, 8\}}$.

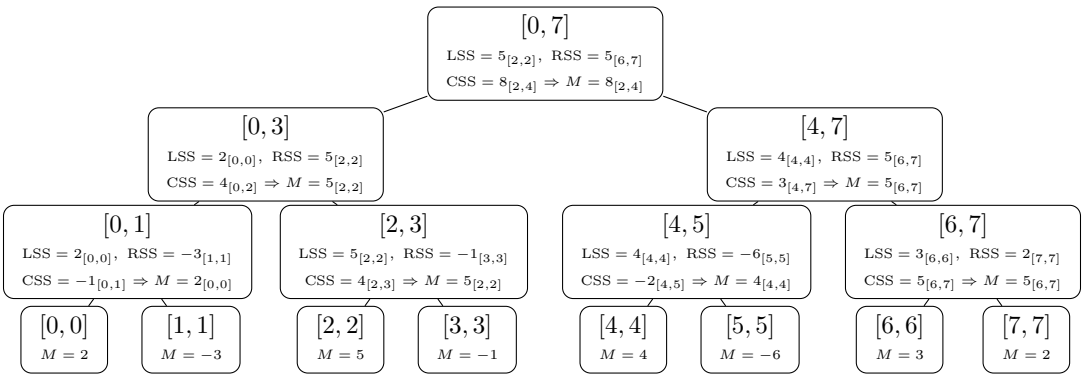
(c) Increasing-order BFS queue:

$$[4] \to [1, 5] \to [5, 2, 6] \to [2, 6, 7] \to [6, 7, 3].$$

$p(1) = 4, p(6) = 1$, hence $\boxed{4 \to 1 \to 6}$ (length 2).

Question 3

At every node, LSS is the best subarray in the left half, RSS is the best in the right half, CSS is the best crossing subarray, and $M = \max\{\text{LSS}, \text{RSS}, \text{CSS}\}$. Subscripts are zero-based selected index ranges.
$A = [2, -3, 5, -1, 4, -6, 3, 2]$.



At the root,

$$\text{CSS} = (5 - 1) + (4) = 8, \qquad \boxed{[5, -1, 4] \text{ gives maximum gain } 8}.$$

$T(N) = 2T(N/2) + \Theta(N) = \Theta(N \log N)$.

Set B

Question 2

(a)

1	$5, 7, 8$	5	$1, 2, 4$
2	$5, 6, 7$	6	$2, 3$
3	$6, 8$	7	$1, 2$
4	$5, 8$	8	$1, 3, 4$

(b) Bipartite: $\boxed{L = \{1, 2, 3, 4\},\ R = \{5, 6, 7, 8\}}$.

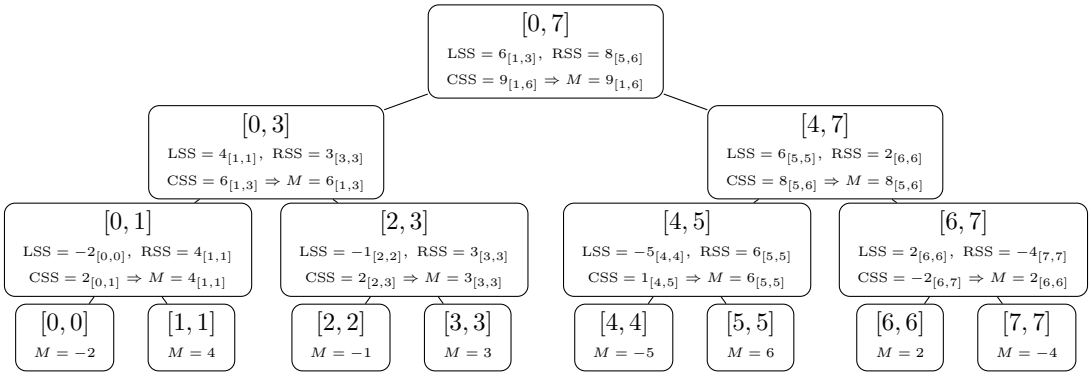
(c) Increasing-order BFS queue:

$$[4] \to [5, 8] \to [8, 1, 2] \to [1, 2, 3] \to [2, 3, 7] \to [3, 7, 6].$$

$p(5) = 4, p(2) = 5, p(6) = 2$, hence $\boxed{4 \to 5 \to 2 \to 6}$ (length 3).

Question 3

The notation is the same as in Set A.
$A = [-2, 4, -1, 3, -5, 6, 2, -4]$.



At the root,

$$\text{CSS} = (4 - 1 + 3) + (-5 + 6 + 2) = 6 + 3 = 9,$$

so $\boxed{[4, -1, 3, -5, 6, 2] \text{ gives maximum gain } 9}$.

CSE221: Algorithms
Quiz 3 Solutions

Sets A and B

Summer 2026

1 Set A

1.1 Question 1: Kosaraju and Topological Ordering [8]

$V = \{A, B, C, D, E, F, G\}, \quad E = \{(A,B), (B,C), (C,A), (C,D), (D,E), (E,D), (E,F), (F,G)\}.$

(a) Kosaraju simulation [5] Original adjacency lists:

A	B	B	C
C	A, D	D	E
E	D, F	F	G
G	$\varnothing$		

First DFS from A:

$$A \to B \to C \to D \to E \to F \to G.$$

$$\boxed{\text{Finish order: } G, F, E, D, C, B, A}$$

Stack bottom $[G, F, E, D, C, B, A]$ top, $\boxed{\text{pop order: } A, B, C, D, E, F, G}$.

Transpose adjacency lists:

A	C	B	A
C	B	D	C, E
E	D	F	E
G	F		

Pop	Transpose DFS	SCC
A	$A \to C \to B$	$\{A, B, C\}$
B, C	already visited	–
D	$D \to E$	$\{D, E\}$
E	already visited	–
F	E already visited	$\{F\}$
G	F already visited	$\{G\}$

(b) SCCs and condensation graph [2]

$$C_1 = \{A, B, C\}, \quad C_2 = \{D, E\}, \quad C_3 = \{F\}, \quad C_4 = \{G\}.$$

$$\boxed{C_1 \longrightarrow C_2 \longrightarrow C_3 \longrightarrow C_4}$$

(c) Topological order [1]

$$\boxed{\{A, B, C\},\ \{D, E\},\ \{F\},\ \{G\}}$$

1.2 Question 2: Dijkstra [7]

$$E = \{(S, A, 4), (S, B, 2), (B, A, 1), (A, C, 3), (B, D, 4), \\ (D, C, 1), (C, T, 2), (D, T, 5)\}.$$

(a) Simulation [4] Initially, $d[S] = 0$; all other distances are ∞.

Step	Extract	Successful relaxation(s)	Active queue
1	$S(0)$	$d[A] = 4, \pi[A] = S;\ d[B] = 2, \pi[B] = S$	$\{(B, 2), (A, 4)\}$
2	$B(2)$	$d[A] = 3, \pi[A] = B;\ d[D] = 6, \pi[D] = B$	$\{(A, 3), (D, 6)\}$
3	$A(3)$	$d[C] = 6, \pi[C] = A$	$\{(C, 6), (D, 6)\}$
4	$C(6)$	$d[T] = 8, \pi[T] = C$	$\{(D, 6), (T, 8)\}$
5	$D(6)$	No update: $6 + 1 > 6,\ 6 + 5 > 8$	$\{(T, 8)\}$
6	$T(8)$	None	$\varnothing$

(b) Final distances and parents [2]

Vertex	S	A	B	C	D	T
Distance	0	3	2	6	6	8
Parent	–	B	S	A	B	C

(c) Shortest path [1]

$$\boxed{S \to B \to A \to C \to T}, \qquad \boxed{\text{cost} = 2 + 1 + 3 + 2 = 8}.$$

1.3 Question 3: Kruskal with DSU [5]

$$E = \{(1, 2, 3), (1, 3, 1), (2, 3, 2), (2, 4, 4), (3, 4, 5), \\ (3, 5, 6), (4, 5, 2), (4, 6, 7), (5, 6, 4)\}.$$

Initial components:

$$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}.$$

Order	Edge	Decision	Components after processing
1	$(1, 3, 1)$	**Accept**	$\{1, 3\}, \{2\}, \{4\}, \{5\}, \{6\}$
2	$(2, 3, 2)$	**Accept**	$\{1, 2, 3\}, \{4\}, \{5\}, \{6\}$
3	$(4, 5, 2)$	**Accept**	$\{1, 2, 3\}, \{4, 5\}, \{6\}$
4	$(1, 2, 3)$	**Reject**	Same root
5	$(2, 4, 4)$	**Accept**	$\{1, 2, 3, 4, 5\}, \{6\}$
6	$(5, 6, 4)$	**Accept**	$\{1, 2, 3, 4, 5, 6\}$
7	$(3, 4, 5)$	**Reject**	Same root
8	$(3, 5, 6)$	**Reject**	Same root
9	$(4, 6, 7)$	**Reject**	Same root

$$\boxed{\text{MST} = \{(1, 3, 1), (2, 3, 2), (4, 5, 2), (2, 4, 4), (5, 6, 4)\}}$$

$$\boxed{\text{Total cost} = 1 + 2 + 2 + 4 + 4 = 13}$$

2 Set B

2.1 Question 1: Kosaraju and Topological Ordering [8]

$V = \{P, Q, R, S, T, U, V\}, \quad E = \{(P,Q), (Q,P), (Q,R), (R,S), (S,T), (T,R), (T,U), (P,V)\}.$

(a) Kosaraju simulation [5] Original adjacency lists:

P	Q, V	Q	P, R
R	S	S	T
T	R, U	U	$\varnothing$
V	$\varnothing$		

First DFS:

$$P \to Q \to R \to S \to T \to U, \quad \text{then } V.$$

$$\boxed{\text{Finish order: } U, T, S, R, Q, V, P}$$

Stack bottom $[U, T, S, R, Q, V, P]$ top, $\boxed{\text{pop order: } P, V, Q, R, S, T, U}$.

Transpose adjacency lists:

P	Q	Q	P
R	Q, T	S	R
T	S	U	T
V	P		

Pop	Transpose DFS	SCC
P	$P \to Q$	$\{P, Q\}$
V	P already visited	$\{V\}$
Q	already visited	–
R	$R \to T \to S$	$\{R, S, T\}$
S, T	already visited	–
U	T already visited	$\{U\}$

(b) SCCs and condensation graph [2]

$$D_1 = \{P, Q\}, \quad D_2 = \{R, S, T\}, \quad D_3 = \{U\}, \quad D_4 = \{V\}.$$

$$\boxed{D_1 \to D_2 \to D_3, \qquad D_1 \to D_4}$$

(c) Topological order [1]

$$\boxed{\{P, Q\},\ \{R, S, T\},\ \{U\},\ \{V\}}$$

2.2 Question 2: Dijkstra [7]

$$E = \{(S, A, 2), (S, B, 5), (A, B, 1), (A, C, 4), (B, D, 2),$$
$$(C, D, 1), (C, T, 3), (D, T, 2)\}.$$

(a) Simulation [4] Initially, $d[S] = 0$; all other distances are ∞.

Step	Extract	Successful relaxation(s)	Active queue
1	$S(0)$	$d[A] = 2, \pi[A] = S;\ d[B] = 5, \pi[B] = S$	$\{(A,2),(B,5)\}$
2	$A(2)$	$d[B] = 3, \pi[B] = A;\ d[C] = 6, \pi[C] = A$	$\{(B,3),(C,6)\}$
3	$B(3)$	$d[D] = 5, \pi[D] = B$	$\{(D,5),(C,6)\}$
4	$D(5)$	$d[T] = 7, \pi[T] = D$	$\{(C,6),(T,7)\}$
5	$C(6)$	No update: $6+1>5,\ 6+3>7$	$\{(T,7)\}$
6	$T(7)$	None	$\varnothing$

(b) Final distances and parents [2]

Vertex	S	A	B	C	D	T
Distance	0	2	3	6	5	7
Parent	–	S	A	A	B	D

(c) Shortest path [1]

$$\boxed{S \to A \to B \to D \to T}, \qquad \boxed{\text{cost} = 2+1+2+2 = 7}.$$

2.3 Question 3: Kruskal with DSU [5]

$$E = \{(1,2,4),(1,3,2),(2,3,1),(2,4,3),(3,5,5),\\ (4,5,2),(4,6,6),(5,6,3),(2,5,7)\}.$$

Initial components:

$$\{1\},\{2\},\{3\},\{4\},\{5\},\{6\}.$$

Order	Edge	Decision	Components after processing
1	$(2,3,1)$	**Accept**	$\{1\},\{2,3\},\{4\},\{5\},\{6\}$
2	$(1,3,2)$	**Accept**	$\{1,2,3\},\{4\},\{5\},\{6\}$
3	$(4,5,2)$	**Accept**	$\{1,2,3\},\{4,5\},\{6\}$
4	$(2,4,3)$	**Accept**	$\{1,2,3,4,5\},\{6\}$
5	$(5,6,3)$	**Accept**	$\{1,2,3,4,5,6\}$
6	$(1,2,4)$	**Reject**	Same root
7	$(3,5,5)$	**Reject**	Same root
8	$(4,6,6)$	**Reject**	Same root
9	$(2,5,7)$	**Reject**	Same root

$$\boxed{\text{MST} = \{(2,3,1),(1,3,2),(4,5,2),(2,4,3),(5,6,3)\}}$$

$$\boxed{\text{Total cost} = 1+2+2+3+3 = 11}$$

CSE221: Algorithms — Quiz 04 Solutions (Summer 2026)

Set A Solutions

Question 1 [10 Marks]

(a) Huffman Coding [3 Marks]

Given Frequencies: $A:5,\ B:9,\ C:12,\ D:13,\ E:16,\ F:45$.

1. Combine $A(5)$ and $B(9) \rightarrow N_1(14)$. Leaves remaining: $C(12), D(13), N_1(14), E(16), F(45)$.
2. Combine $C(12)$ and $D(13) \rightarrow N_2(25)$. Remaining: $N_1(14), E(16), N_2(25), F(45)$.
3. Combine $N_1(14)$ and $E(16) \rightarrow N_3(30)$. Remaining: $N_2(25), N_3(30), F(45)$.
4. Combine $N_2(25)$ and $N_3(30) \rightarrow N_4(55)$. Remaining: $F(45), N_4(55)$.
5. Combine $F(45)$ and $N_4(55) \rightarrow$ Root(100).

Codewords (assigning left edge = 0, right edge = 1):

Character	Frequency (f)	Codeword	Length (l)
F	45	`0`	1
C	12	`100`	3
D	13	`101`	3
E	16	`111`	3
A	5	`1100`	4
B	9	`1101`	4

Total Encoded Length:

$$\begin{aligned} L &= \sum (f_i \times l_i) \\ &= (45 \times 1) + (12 \times 3) + (13 \times 3) + (16 \times 3) + (5 \times 4) + (9 \times 4) \\ &= 45 + 36 + 39 + 48 + 20 + 36 = \mathbf{224} \text{ bits.} \end{aligned}$$

(b) Fractional Knapsack [2 Marks]

Capacity: $W = 50$. **Items** (w_i, p_i)**:** $I_1(10, 60), I_2(20, 100), I_3(30, 120)$.

- Profit-to-weight ratios:

$$r_1 = \frac{60}{10} = 6.0, \quad r_2 = \frac{100}{20} = 5.0, \quad r_3 = \frac{120}{30} = 4.0$$

- Order of selection: $I_1 \succ I_2 \succ I_3$.
- Take all of I_1: Weight = 10, Profit = 60, Remaining capacity = $50 - 10 = 40$.
- Take all of I_2: Weight = 20, Profit = 100, Remaining capacity = $40 - 20 = 20$.
- Take fraction $\frac{20}{30} = \frac{2}{3}$ of I_3: Weight = 20, Profit = $\frac{2}{3} \times 120 = 80$.

Maximum Profit: $60 + 100 + 80 =$ **240**.

(c) 0/1 Knapsack [5 Marks]

Capacity: $W = 7$. **Weights:** $[1, 3, 4, 5]$; **Values:** $[1, 4, 5, 7]$.

$$V[i, w] = \begin{cases} V[i-1, w] & \text{if } w_i > w \\ \max(V[i-1, w],\ v_i + V[i-1, w - w_i]) & \text{if } w_i \leq w \end{cases}$$

$i \setminus w$	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
0	0	0	0	0	0	0	0	0
1 ($w_1 = 1, v_1 = 1$)	0	1	1	1	1	1	1	1
2 ($w_2 = 3, v_2 = 4$)	0	1	1	4	5	5	5	5
3 ($w_3 = 4, v_3 = 5$)	0	1	1	4	5	6	6	**9**
4 ($w_4 = 5, v_4 = 7$)	0	1	1	4	5	7	8	**9**

Traceback:

- $V[4, 7] = V[3, 7] = 9 \implies$ Item 4 is **not selected**.
- $V[3, 7] \neq V[2, 7]$ $(9 \neq 5) \implies$ **Item 3 is selected**; remaining weight $= 7 - 4 = 3$.
- $V[2, 3] \neq V[1, 3]$ $(4 \neq 1) \implies$ **Item 2 is selected**; remaining weight $= 3 - 3 = 0$.

Maximum Value: 9
Selected Items: Item 2 and Item 3 (weights 3 and 4).

Question 2 [05 Marks]

(a) 15-Puzzle [2 Marks]

- **Start configuration:** Blank is at row 4, column 3 (position between 14 and 15).
- **Goal configuration:** Blank is at row 4, column 4.
- **Search order:** U, D, L, R with depth limit 2 (no immediate reverse).
- **Exploration from root (depth 0):**
  1. Try **U**: Blank moves to $(3, 3)$ (depth 1). Its valid next moves (avoiding reverse D) at depth 2 (U, L, R) do not reach the goal. Backtrack to root.
  2. Try **D**: Out of bounds (blank is in bottom row).
  3. Try **L**: Blank moves to $(4, 2)$ (depth 1). Valid next moves at depth 2 (U, L) do not reach the goal. Backtrack to root.
  4. Try **R**: Blank swaps with 15 and moves to $(4, 4)$ (depth 1). This matches the goal state!

Successful Move Sequence: R (Single move to the Right).

(b) & (c) 4-Queens Problem (Ascending Order 1 to 4) [3 Marks]

Place one queen per row (r_1, r_2, r_3, r_4):

1. $[1]$
2. $[1,1]$ (conflict), $[1,2]$ (diagonal conflict)
3. $[1,3]$ (valid)
4. $[1,3,1]$ (conflict), $[1,3,2]$ (conflict), $[1,3,3]$ (conflict), $[1,3,4]$ (conflict) $\rightarrow$ backtrack
5. $[1,4]$ (valid)
6. $[1,4,1]$ (conflict)
7. $[1,4,2]$ (valid)
8. $[1,4,2,1]$ (conflict), $[1,4,2,2]$ (conflict), $[1,4,2,3]$ (conflict), $[1,4,2,4]$ (conflict) $\rightarrow$ backtrack
9. $[1,4,3]$ (conflict), $[1,4,4]$ (conflict) $\rightarrow$ backtrack to row 1
10. $[2]$ (valid)
11. $[2,1]$ (conflict), $[2,2]$ (conflict), $[2,3]$ (conflict)
12. $[2,4]$ (valid)
13. $[2,4,1]$ (valid)
14. $[2,4,1,1]$ (conflict), $[2,4,1,2]$ (conflict)
15. $[2,4,1,3]$ (valid — all 4 queens placed!)

Solution: $[c_1, c_2, c_3, c_4] = [\mathbf{2}, \mathbf{4}, \mathbf{1}, \mathbf{3}]$.

Question 3 [05 Marks]

LCS of $X =$ ABCBDAB and $Y =$ BDCABA:

$L[i,j]$	$\emptyset$	**B**	**D**	**C**	**A**	**B**	**A**
$\emptyset$	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
B	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
B	0	1	1	2	2	3	3
D	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
B	0	1	2	2	3	4	**4**

Traceback:

- $(7,6)$ [B vs A]: $L[6,6] = 4 \rightarrow$ go up to $(6,6)$.
- $(6,6)$ [A vs A]: Match! Take **A**, move diagonally to $(5,5)$.
- $(5,5)$ [D vs B]: $L[4,5] = 3 \rightarrow$ go up to $(4,5)$.

- $(4, 5)$ [B vs B]: Match! Take **B**, move diagonally to $(3, 4)$.
- $(3, 4)$ [C vs A]: $L[3, 3] = 2 \rightarrow$ go left to $(3, 3)$.
- $(3, 3)$ [C vs C]: Match! Take **C**, move diagonally to $(2, 2)$.
- $(2, 2)$ [B vs D]: $L[2, 1] = 1 \rightarrow$ go left to $(2, 1)$.
- $(2, 1)$ [B vs B]: Match! Take **B**, move diagonally to $(1, 0)$.

Longest Common Subsequence Length: 4
LCS: BCBA (or equivalently BDAB / BCAB).

Set B Solutions

Question 1 [10 Marks]

(a) Huffman Coding [3 Marks]

Given Frequencies: $P:5,\ Q:7,\ R:10,\ S:15,\ T:20,\ U:45$.

1. Combine $P(5)$ and $Q(7) \to N_1(12)$. Remaining: $R(10), N_1(12), S(15), T(20), U(45)$.
2. Combine $R(10)$ and $N_1(12) \to N_2(22)$. Remaining: $S(15), T(20), N_2(22), U(45)$.
3. Combine $S(15)$ and $T(20) \to N_3(35)$. Remaining: $N_2(22), N_3(35), U(45)$.
4. Combine $N_2(22)$ and $N_3(35) \to N_4(57)$. Remaining: $U(45), N_4(57)$.
5. Combine $U(45)$ and $N_4(57) \to$ Root(102).

Codewords (assigning left edge = 0, right edge = 1):

Character	**Frequency (f)**	**Codeword**	**Length (l)**
U	45	`0`	1
S	15	`110`	3
T	20	`111`	3
R	10	`100`	3
P	5	`1010`	4
Q	7	`1011`	4

Total Encoded Length:

$$\begin{aligned} L &= \sum (f_i \times l_i) \\ &= (45 \times 1) + (15 \times 3) + (20 \times 3) + (10 \times 3) + (5 \times 4) + (7 \times 4) \\ &= 45 + 45 + 60 + 30 + 20 + 28 = \mathbf{228} \text{ bits.} \end{aligned}$$

(b) Fractional Knapsack [2 Marks]

Capacity: $W = 40$. **Items** (w_i, p_i)**:** $I_1(10, 60), I_2(20, 110), I_3(30, 120)$.

- Profit-to-weight ratios:

$$r_1 = \frac{60}{10} = 6.0, \quad r_2 = \frac{110}{20} = 5.5, \quad r_3 = \frac{120}{30} = 4.0$$

- Order of selection: $I_1 \succ I_2 \succ I_3$.
- Take all of I_1: Weight = 10, Profit = 60, Remaining capacity = $40 - 10 = 30$.
- Take all of I_2: Weight = 20, Profit = 110, Remaining capacity = $30 - 20 = 10$.
- Take fraction $\frac{10}{30} = \frac{1}{3}$ of I_3: Weight = 10, Profit = $\frac{1}{3} \times 120 = 40$.

Maximum Profit: $60 + 110 + 40 = \mathbf{210}$.

(c) 0/1 Knapsack [5 Marks]

Capacity: $W = 8$. **Weights:** $[2, 3, 4, 5]$; **Values:** $[3, 4, 5, 8]$.

$i \setminus w$	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
0	0	0	0	0	0	0	0	0	0
1 ($w_1 = 2, v_1 = 3$)	0	0	3	3	3	3	3	3	3
2 ($w_2 = 3, v_2 = 4$)	0	0	3	4	4	7	7	7	7
3 ($w_3 = 4, v_3 = 5$)	0	0	3	4	5	7	8	9	9
4 ($w_4 = 5, v_4 = 8$)	0	0	3	4	5	8	8	11	**12**

Traceback:

- $V[4, 8] = 12 \neq V[3, 8] = 9 \implies$ **Item 4 is selected**; remaining weight $= 8 - 5 = 3$.
- $w_3 = 4 > 3 \implies$ Item 3 cannot be selected.
- $V[2, 3] = 4 \neq V[1, 3] = 3 \implies$ **Item 2 is selected**; remaining weight $= 3 - 3 = 0$.

Maximum Value: 12
Selected Items: Item 2 and Item 4 (weights 3 and 5).

Question 2 [05 Marks]

(a) 15-Puzzle [2 Marks]

- **Start configuration:** Blank is at row 3, column 3 (between 10 and 12, with 11 below it).
- **Goal configuration:** Blank is at row 4, column 4.
- **Search order:** U, D, L, R with depth limit 2 (no immediate reverse).
- **Exploration from root (depth 0):**
  1. Try **U**: Blank moves to $(2, 3)$ (depth 1). Branches at depth 2 do not lead to the goal. Backtrack.
  2. Try **D**: Blank swaps with 11 and moves to $(4, 3)$ (depth 1). Row 3 is now $[9, 10, 11, 12]$ and Row 4 is $[13, 14, _, 15]$.
     - From this state, try moves in order (avoiding U):
     - Try D: Out of bounds.
     - Try L: Blank swaps with 14 (depth 2, not goal).
     - Try **R**: Blank swaps with 15 to reach position $(4, 4)$ (depth 2). This matches the goal state!

Successful Move Sequence: D → R (Down, then Right).

(b) & (c) 4-Queens Problem (Descending Order 4 to 1) [3 Marks]

Place one queen per row (r_1, r_2, r_3, r_4):

1. $[4]$
2. $[4, 4]$ (conflict), $[4, 3]$ (diagonal conflict)

3. $[4, 2]$ (valid)
4. $[4, 2, 4]$ (conflict), $[4, 2, 3]$ (conflict), $[4, 2, 2]$ (conflict), $[4, 2, 1]$ (conflict) $\rightarrow$ backtrack
5. $[4, 1]$ (valid)
6. $[4, 1, 4]$ (conflict)
7. $[4, 1, 3]$ (valid)
8. $[4, 1, 3, 4]$ (conflict), $[4, 1, 3, 3]$ (conflict), $[4, 1, 3, 2]$ (conflict), $[4, 1, 3, 1]$ (conflict) $\rightarrow$ backtrack
9. $[4, 1, 2]$ (conflict), $[4, 1, 1]$ (conflict) $\rightarrow$ backtrack to row 1
10. $[3]$ (valid)
11. $[3, 4]$ (conflict), $[3, 3]$ (conflict), $[3, 2]$ (conflict)
12. $[3, 1]$ (valid)
13. $[3, 1, 4]$ (valid)
14. $[3, 1, 4, 4]$ (conflict), $[3, 1, 4, 3]$ (conflict)
15. $[3, 1, 4, 2]$ (valid — all 4 queens placed!)

Solution: $[c_1, c_2, c_3, c_4] = [\mathbf{3}, \mathbf{1}, \mathbf{4}, \mathbf{2}]$.

Question 3 [05 Marks]

LCS of $X =$ XMJYAUZ and $Y =$ MZJAWXU:

$L[i,j]$	$\emptyset$	**M**	**Z**	**J**	**A**	**W**	**X**	**U**
$\emptyset$	0	0	0	0	0	0	0	0
X	0	0	0	0	0	0	1	1
M	0	1	1	1	1	1	1	1
J	0	1	1	2	2	2	2	2
Y	0	1	1	2	2	2	2	2
A	0	1	1	2	3	3	3	3
U	0	1	1	2	3	3	3	4
Z	0	1	2	2	3	3	3	**4**

Traceback:

- $(7, 7)$ [Z vs U]: $L[6, 7] = 4 \rightarrow$ go up to $(6, 7)$.
- $(6, 7)$ [U vs U]: Match! Take **U**, move diagonally to $(5, 6)$.
- $(5, 6)$ [A vs X]: $L[5, 5] = 3 \rightarrow$ go left to $(5, 5)$.
- $(5, 5)$ [A vs W]: $L[5, 4] = 3 \rightarrow$ go left to $(5, 4)$.
- $(5, 4)$ [A vs A]: Match! Take **A**, move diagonally to $(4, 3)$.
- $(4, 3)$ [Y vs J]: $L[3, 3] = 2 \rightarrow$ go up to $(3, 3)$.
- $(3, 3)$ [J vs J]: Match! Take **J**, move diagonally to $(2, 2)$.

- $(2, 2)$ [M vs Z]: $L[2, 1] = 1 \rightarrow$ go left to $(2, 1)$.
- $(2, 1)$ [M vs M]: Match! Take **M**, move diagonally to $(1, 0)$.

Longest Common Subsequence Length: 4
LCS: MJAU.

CSE221: Algorithms — Quiz 05 Solutions (Summer 2026)

Question 1 [05 Marks]

(a) Kosaraju's Algorithm Simulation [3 Marks]

Graph: $V = \{A, B, C, D, E, F, G\}$
$E = \{(A,B), (B,C), (C,A), (B,D), (D,E), (E,D), (C,F), (F,G), (E,G)\}$

- **First DFS (original graph, starting at A, tie-breaking alphabetically):**
  1. Start at $A \to B \to C \to F \to G$.
  2. G has no unvisited neighbors $\implies$ finishes at $t = 6$.
  3. Backtrack to $F \implies$ finishes at $t = 7$.
  4. Backtrack to C (neighbor A already visited) $\implies$ finishes at $t = 8$.
  5. Backtrack to $B \to D \to E$.
  6. From E, neighbor D is visited; neighbor G is visited $\implies$ E finishes at $t = 11$.
  7. Backtrack to $D \implies$ finishes at $t = 12$.
  8. Backtrack to $B \implies$ finishes at $t = 13$.
  9. Backtrack to $A \implies$ finishes at $t = 14$.

  Finishing Times: $A : 14,\ B : 13,\ D : 12,\ E : 11,\ C : 8,\ F : 7,\ G : 6$.

- **Order used in Second DFS (descending finish time):**

$$A \longrightarrow B \longrightarrow D \longrightarrow E \longrightarrow C \longrightarrow F \longrightarrow G$$

- **Second DFS (on reversed graph G^T):**
  1. Run DFS from A: visits $\{A, C, B\} \implies \text{SCC}_{\mathbf{1}} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$.
  2. Next unvisited is D: visits $\{D, E\} \implies \text{SCC}_{\mathbf{2}} = \{\mathbf{D}, \mathbf{E}\}$.
  3. Next unvisited is F: visits $\{F\} \implies \text{SCC}_{\mathbf{3}} = \{\mathbf{F}\}$.
  4. Next unvisited is G: visits $\{G\} \implies \text{SCC}_{\mathbf{4}} = \{\mathbf{G}\}$.

(b) Condensation Graph & Topological Order [1 Mark]

Treating each SCC as a super-node:

- $\text{SCC}_1 = \{A, B, C\}$, $\text{SCC}_2 = \{D, E\}$, $\text{SCC}_3 = \{F\}$, $\text{SCC}_4 = \{G\}$.

- Directed edges between super-nodes:

$$\text{SCC}_1 \longrightarrow \text{SCC}_2 \quad (\text{via } (B, D)), \quad \text{SCC}_1 \longrightarrow \text{SCC}_3 \quad (\text{via } (C, F))$$

$$\text{SCC}_2 \longrightarrow \text{SCC}_4 \quad (\text{via } (E, G)), \quad \text{SCC}_3 \longrightarrow \text{SCC}_4 \quad (\text{via } (F, G))$$

- **Valid Topological Order of Super-Nodes:**

$$\text{SCC}_{\mathbf{1}} \longrightarrow \text{SCC}_{\mathbf{2}} \longrightarrow \text{SCC}_{\mathbf{3}} \longrightarrow \text{SCC}_{\mathbf{4}} \quad (\text{or } \text{SCC}_1 \to \text{SCC}_3 \to \text{SCC}_2 \to \text{SCC}_4)$$

(c) Minimum Routes Closed for DAG / Topological Order [1 Mark]

- A directed graph has a topological order if and only if it is a Directed Acyclic Graph (DAG).
- The original graph contains two directed cycles: $A \to B \to C \to A$ and $D \to E \to D$.
- Since these cycles are completely vertex-disjoint, closing at least 1 edge from each cycle is strictly necessary and sufficient.
- **Minimum routes to close: 2** routes.
- **Routes to close:** $(\mathbf{C}, \mathbf{A})$ and $(\mathbf{E}, \mathbf{D})$ (or any one edge from $\{(A, B), (B, C), (C, A)\}$ and one from $\{(D, E), (E, D)\}$).

Question 2 [05 Marks]

(a) Interval Scheduling (Greedy) [1 Mark]

- **Rule Name: Earliest Finish Time First (EFTF)**.
- Sort intervals by non-decreasing finish time: $W_1(1, 4)$, $W_2(2, 6)$, $W_3(4, 7)$, $W_4(5, 9)$, $W_5(7, 10)$, $W_6(8, 12)$, $W_7(10, 13)$.
- **Greedy Selection:**
  1. Select $W_1(1, 4)$.
  2. First compatible is $W_3(4, 7)$ (since $4 \geq 4$).
  3. First compatible is $W_5(7, 10)$ (since $7 \geq 7$).
  4. First compatible is $W_7(10, 13)$ (since $10 \geq 10$).
- **Selected Workshops:** $\{\mathbf{W_1}, \mathbf{W_3}, \mathbf{W_5}, \mathbf{W_7}\}$ (Maximum count = **4**).

(b) Weighted Interval Scheduling [1 Mark]

- Total value of schedule from (a): Value(W_1) + Value(W_3) + Value(W_5) + Value(W_7) = $2 + 2 + 2 + 3 = \mathbf{9}$ Tk.
- **Alternative Feasible Schedule:** Select $\{W_1, W_4, W_7\}$.
  - Intervals $(1, 4)$, $(5, 9)$, and $(10, 13)$ do not overlap.
  - Total Value: $2 + 10 + 3 = \mathbf{15}$ Tk > 9 Tk.
- **Conclusion:** The greedy Earliest Finish Time rule does **not** guarantee an optimal solution for the weighted interval scheduling problem; dynamic programming is required.

(c) Huffman Coding [3 Marks]

Given Frequencies: $D:5,\ B:10,\ C:14,\ F:16,\ A:25,\ E:30$. Total characters = 100.

1. Combine $D(5)$ and $B(10) \rightarrow N_1(15)$. Remaining: $C(14), N_1(15), F(16), A(25), E(30)$.
2. Combine $C(14)$ and $N_1(15) \rightarrow N_2(29)$. Remaining: $F(16), A(25), N_2(29), E(30)$.
3. Combine $F(16)$ and $A(25) \rightarrow N_3(41)$. Remaining: $N_2(29), E(30), N_3(41)$.
4. Combine $N_2(29)$ and $E(30) \rightarrow N_4(59)$. Remaining: $N_3(41), N_4(59)$.
5. Combine $N_3(41)$ and $N_4(59) \rightarrow \text{Root}(100)$.

Codewords (left edge = 0, right edge = 1):

Character	**Frequency (f)**	**Codeword**	**Length (l)**
F	16	`00`	2
A	25	`01`	2
E	30	`11`	2
C	14	`100`	3
D	5	`1010`	4
B	10	`1011`	4

- **Huffman Encoded Length:**

  $(16\times2)+(25\times2)+(30\times2)+(14\times3)+(5\times4)+(10\times4) = 32+50+60+42+20+40 =$ **244** bits

- **Fixed-Length Code:** For 6 distinct characters, $\lceil \log_2 6 \rceil = 3$ bits/char.

  $$\text{Total Length} = 100 \times 3 = \textbf{300} \text{ bits}$$

- **Bits Saved:** $300 - 244 =$ **56** bits.

Question 3 [04 Marks]

(a) 15-Puzzle [1 Mark]

- **Initial Configuration:** Blank is at row 3, column 2. Tile 10 is at $(4,2)$, tile 14 is at $(4,3)$, and tile 15 is at $(4,4)$.
- **Shortest Move Sequence:**

  $$\text{Down} \longrightarrow \text{Right} \longrightarrow \text{Right} \quad (\mathbf{D} \rightarrow \mathbf{R} \rightarrow \mathbf{R})$$

  1. **Down:** Blank swaps with $10 \rightarrow$ row 3 becomes $[9, 10, 11, 12]$, row 4 becomes $[13, _, 14, 15]$.
  2. **Right:** Blank swaps with $14 \rightarrow$ row 4 becomes $[13, 14, _, 15]$.
  3. **Right:** Blank swaps with $15 \rightarrow$ row 4 becomes $[13, 14, 15, _]$ (Goal State).

- **Backtracking Condition:** Check that the move does not go **out of bounds**, does not **immediately reverse** the parent move, or does not exceed the allowed **depth bound** / repeat an ancestor state.

(b) Fractional and 0/1 Knapsack ($W = 10$ kg) [2 Marks]

Parcels: $P_1(7, 42) \implies r_1 = 6.0$; $P_2(5, 25) \implies r_2 = 5.0$; $P_3(4, 16) \implies r_3 = 4.0$; $P_4(3, 9) \implies r_4 = 3.0$.

- **Fractional Knapsack:**
  - Take all of P_1 (7 kg, profit 42). Remaining capacity = 3 kg.
  - Take fraction $\frac{3}{5}$ of P_2 (3 kg, profit $\frac{3}{5} \times 25 = 15$).
  - **Maximum Profit:** $42 + 15 = \mathbf{57}$ Tk.
  - **Parcels Taken:** All of P_1 and $\frac{3}{5}$ of P_2.
- **0/1 Knapsack:**
  - Possible subset combinations within 10 kg: $\{P_1, P_4\} \implies w = 7 + 3 = 10$, profit $= 42 + 9 = \mathbf{51}$ Tk.
    $\{P_2, P_3\} \implies w = 5 + 4 = 9$, profit $= 25 + 16 = 41$ Tk.
    $\{P_2, P_4\} \implies w = 5 + 3 = 8$, profit $= 25 + 9 = 34$ Tk.
  - **Maximum Profit: 51** Tk.
  - **Parcels Taken:** $\{\mathbf{P_1}, \mathbf{P_4}\}$.

(c) LCS for $X =$ ACBD and $Y =$ ABCD [1 Mark]

- **Recurrence Relation:**

$$L[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L[i-1, j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(L[i-1, j],\ L[i, j-1]) & \text{if } X[i] \neq Y[j] \end{cases}$$

- **Length of LCS: 3**.
- **Valid LCS String:** ACD (or ABD).

Question 4 [06 Marks]

(a) Dijkstra's Algorithm from Source A [3 Marks]

Edge List: $(A, B, 6), (A, C, 2), (B, C, 5), (B, D, 3), (C, D, 8), (C, E, 4), (D, E, 9), (D, F, 7), (E, F, 1), (E, G, 12), ($

Step	Extract Min	Distance Updates / Edge Relaxations	Priority Queue (unvisit
0	—	Init: $d[A] = 0$, all others $= \infty$	$\{(A, 0)\}$
1	A $(d = 0)$	$d[C] = \min(\infty, 0 + 2) = 2$, $d[B] = \min(\infty, 0 + 6) = 6$	$\{(C, 2), (B, 6)\}$
2	C $(d = 2)$	$d[B] = \min(6, 2 + 5) = 6$, $d[E] = \min(\infty, 2 + 4) = 6$, $d[D] = \min(\infty, 2 + 8) = 10$	$\{(B, 6), (E, 6), (D, 10)\}$
3	B $(d = 6)$	$d[D] = \min(10, 6 + 3) = 9$	$\{(E, 6), (D, 9)\}$
4	E $(d = 6)$	$d[F] = \min(\infty, 6 + 1) = 7$, $d[G] = \min(\infty, 6 + 12) = 18$, $d[D] = \min(9, 6 + 9) = 9$	$\{(F, 7), (D, 9), (G, 18)\}$
5	F $(d = 7)$	$d[D] = \min(9, 7 + 7) = 9$, $d[G] = \min(18, 7 + 11) = 18$	$\{(D, 9), (G, 18)\}$
6	D $(d = 9)$	No improvements	$\{(G, 18)\}$
7	G $(d = 18)$	Final vertex finalized	$\emptyset$

Final Shortest Distances from A:

$$d[A] = 0, \quad d[B] = 6, \quad d[C] = 2, \quad d[D] = 9, \quad d[E] = 6, \quad d[F] = 7, \quad d[G] = 18.$$

(b) Kruskal's Algorithm Simulation with DSU [3 Marks]

Number of vertices $= 7 \implies$ MST requires $7 - 1 = 6$ edges.
Sorted Edge List and DSU Trace:

Edge	Weight	Find Operations	Action	Reason
(E, F)	1	$\text{Find}(E) \neq \text{Find}(F)$	**Accepted**	Union $\{E, F\}$
(A, C)	2	$\text{Find}(A) \neq \text{Find}(C)$	**Accepted**	Union $\{A, C\}$
(B, D)	3	$\text{Find}(B) \neq \text{Find}(D)$	**Accepted**	Union $\{B, D\}$
(C, E)	4	$\text{Find}(C) \neq \text{Find}(E)$	**Accepted**	Union $\{A, C, E, F\}$
(B, C)	5	$\text{Find}(B) \neq \text{Find}(C)$	**Accepted**	Union $\{A, B, C, D, E, F\}$
(A, B)	6	$\text{Find}(A) = \text{Find}(B)$	**Rejected**	Forms cycle $A - B - C - A$
(D, F)	7	$\text{Find}(D) = \text{Find}(F)$	**Rejected**	Forms cycle
(C, D)	8	$\text{Find}(C) = \text{Find}(D)$	**Rejected**	Forms cycle
(D, E)	9	$\text{Find}(D) = \text{Find}(E)$	**Rejected**	Forms cycle
(F, G)	11	$\text{Find}(F) \neq \text{Find}(G)$	**Accepted**	Connects G (6th edge)
(E, G)	12	$\text{Find}(E) = \text{Find}(G)$	**Rejected**	Forms cycle

- **Selected Links (MST):**

$$(E, F, 1),\ (A, C, 2),\ (B, D, 3),\ (C, E, 4),\ (B, C, 5),\ (F, G, 11)$$

- **Total Minimum Cost:**

$$1 + 2 + 3 + 4 + 5 + 11 = \mathbf{26}.$$